

Team 39 - Navigating forwards within the AI Team

Foreword

- In general, we should outsource our processing and {common, repetitive tasks} to 'libraries' and frameworks.
- Currently, myself and Daniel have been experimenting with Milvus, a solution for vector databases.
- Milvus cannot run standalone and requires Docker to run locally. This could be a potential caveat, inhibiting development and scalability.
- I suggest a pivot in our current technology stack. We could use [Azure AI Search](#) (free tier), through their API.
- This will allow us to perform more intensive searches, and enable us to add additional parameters for the similarity search, such as user preferences.

AI Team Layout

1. **Data Collection**
 - User Profiles:
Employees' email, password, etc, carpooling preferences (non-smokers/smokers, pet owners/non-pet owners)
 - User Journeys:
Home addresses, work addresses, Work schedules (time of departure).
 - Preferences + Interests:
Optional data on interests could help in making it a more enjoyable experience for participants. (This could also be a great way to market the product)
E.g. research data on positive impact that social connections have on people.
2. **Generating Embeddings**
 - Use Azure Search to represent the collected data in a high-dimensional space.
 - This includes converting textual information (like preferences) into vectors that a machine can understand and process.
 - In order to maximise the accuracy of the search, we should use Maps API to enable the location pinpoint functionality to find coordinates of start/end locations.
3. **Similarity Matching**
 - Location and Schedule Matching:
Find matches based on geographical proximity and compatible schedules.
This involves calculating the distance between embeddings of addresses and time availability.
 - Preference and Interest Matching:
Further refine matches by comparing preferences and interests to ensure compatibility beyond just logistical convenience.
4. **Ranking and Suggestions**
 - Once potential matches are identified, rank them based on a combination of factors such as distance, schedule alignment, and preference match scores.
5. **Privacy and Security**
 - Ensure the system adheres to privacy laws and corporate policies. Securely handle personal information, and possibly allow users to opt-in to the service
6. **User Interface**
 - Develop a user-friendly interface that allows employees to easily input their information, receive match suggestions, and communicate with potential carpool partners.

This layout would ensure that users can seamlessly connect with one another and share commutes, leading to a greener outcome.

Use Case Exploration

To further clarify the use case, it would be a good idea to have a holistic view of the system functionality. Here are some notes on potential areas which could be further explored.

Weekly variations in travel

- There should be an option to change your travel plan from day to day, i.e. Monday travel by bus, Tuesday travel by carpooling.
- This increases complexity, however is necessary for a comprehensive system.
- Perhaps we should allow the user to first select their modes of transport for each day of the week. (This should also be editable later on.)
- If they choose 'by car', they enter type of engine, but they should also have an option to tick a 'share commute' checkbox, and enter the number of seats in the car (could autofill with another API), and submitting will place their car inside of a Car table.

Things we should strongly consider in order to achieve high marks:

1. Scalability and Flexibility
2. Management Panel
 - Ensure data privacy/ protection applies -> we could dedicate a team to research and present a short presentation on proper etiquette to ensure our team has comprehensive coverage on this topic.
3. Since this software is used by large corporations, they will pay for this software and we should consider how the data migration (or lack of) should work, in accordance with data privacy, and potential employees wanting to opt-out of the system.
4. Feedback Loop
 - Incorporate a feedback system where users can rate their carpooling experience. Use this data to adjust and improve the matching algorithm over time

Questions

I believe that to fully understand the app which we are producing, we should be asking as many questions as possible. Here are some of my questions for the team:

- What are we going to offer people who already take public transport? A chance to carpool with people who drive anyway?
- How would we store cars for carpooling?
 - To store matches for each car, we would require first a search to match people, but this would need to be restricted in some ways, as the system shouldn't allow for 10 users to carpool all in the same car.
 - Therefore, I propose a car table, in which users would submit their car.
 - Then, when searching the database, you could filter by number of available seats in the car being strictly greater than number of people carpooling on this specific day.
 - Would we also need to create a database for each car? As in the car that is being carpoled. How many people max? 4? 5?
 - Set limit in database?
 - E.g. when user signs up and agrees to take other employees, They should input how many seats available in the car to carpool.
 - Also should there be a situation where user1 wants to carpool with user2 on Monday, but then on Tuesday user2 wants to carpool with user1?
 - As in, they switch to the other persons car, which means car id would change.
- How will the front-end handle this?